❷ Mocks the ReactiveJwtDecoder so that the application doesn't try to call Keycloak and get the public key for decoding the Access Token

❸ Mutates the HTTP request with a mock, JWT-formatted Access Token for a user with the "customer" role

Open a Terminal window, navigate to the Order Service root folder, and run the newly added tests as follows:

```
$ ./gradlew test --tests OrderControllerWebFluxTests
```

As always, you can find more test examples in the source code repository accompanying this book (Chapter12/12-end/order-service).

12.4 Protecting and auditing data with Spring Security and Spring Data

So far, we've looked at securing the APIs exposed by Spring Boot applications and handling concerns like authentication and authorization. What about data? Once you have Spring Security in place, you can also secure the business and data layers.

Regarding the business logic, you can enable the method security feature to check for user authentication or authorization directly on business methods, leveraging annotations like `@PreAuthorize`. In the Polar Bookshop system, the business layer is not complex enough to require additional security policies, so I won't describe that.

NOTE To learn more about how to use method authentication and authorization, refer to chapter 8 of *Spring Security in Action* by Laurenţiu Spilcă (Manning, 2020), where those topics are explained in great detail.